

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	M.KASIF JALAL
Name	192424214

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I M.KASIF JALAL certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: KASIF

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

ANS:

- In Hadoop MapReduce, a job is divided into Map and Reduce phases so that large datasets can be processed in parallel across multiple nodes. The input data is divided into input splits, and each split is generally processed by one Mapper task. The intermediate output generated by the Mappers is then transferred through the Shuffle and Sort phase to the Reducers.
- If a Hadoop job repeatedly fails because input splits are highly uneven and many Reducers remain idle, the problem is usually related to poor input-data organization, inappropriate split configuration, incorrect partitioning, or an imbalance between the number of Map and Reduce tasks. This condition can create a data-skew problem in which some tasks process much more data than others.

Identifying the Likely Design Error

The most probable design error is **poor data partitioning or an inappropriate input-split configuration**.

For example, assume that a dataset contains 1 TB of log data. If the input files are organized poorly, one input split may contain a very large amount of data while other splits contain much smaller amounts.

The task distribution might look like:

- Mapper 1 → 10 GB
- Mapper 2 → 10 GB
- Mapper 3 → 11 GB
- Mapper 4 → 200 GB
- Mapper 5 → 12 GB

In this situation, most Mappers finish quickly while Mapper 4 continues processing for a long time. This creates a **straggler task**.

The Reducer problem can also be caused by an unsuitable key distribution. If most intermediate records have the same key, one Reducer may receive most of the data while other Reducers receive little or no data.

Input Split Imbalance

Input splits determine how Map tasks are created. The split size should be selected according to the input data, block size, available CPU cores, and cluster capacity.

If the input consists of many small files, Hadoop may create a large number of small tasks. This creates excessive task scheduling overhead.

On the other hand, extremely large files or unsuitable split settings can reduce parallelism.

Therefore, the input data should be organized into reasonably sized files and partitions.

Reducer Idleness

Reducers receive intermediate data according to the partitioning function.

Normally, Hadoop uses a partitioner that determines which Reducer receives a particular key.

For example:

$\text{hash}(\text{key}) \% \text{number_of_reducers}$

If the keys are distributed evenly, the workload should be reasonably balanced.

However, suppose 90% of the records have the key "India":

India → 90 million records

while other keys have only a few thousand records.

One Reducer may receive most of the records, while other Reducers remain idle.

This is called **data skew**.

Corrective Action 1: Reorganize Input Data

The first corrective action is to reorganize the input data into appropriately sized files.

Large datasets should be divided into manageable HDFS blocks and input splits. The split size should provide sufficient parallelism without creating excessive numbers of Mapper tasks.

For example, instead of having a few extremely large files, the data can be organized into multiple balanced files based on:

- Date
- Region
- Data source
- Application
- Event type

This allows Mappers to process approximately equal amounts of data.

Corrective Action 2: Review HDFS Block and Split Configuration

The HDFS block size and MapReduce input split size should be reviewed.

If the split size is too large, fewer Mapper tasks are created, reducing parallelism.

If it is too small, too many Mapper tasks are generated, increasing scheduling overhead.

Therefore, the split size should be selected based on the cluster size and workload.

Corrective Action 3: Use a Suitable Partitioner

If the Reducer workload is uneven, the partitioning logic should be examined.

The default hash partitioner works well when keys are reasonably distributed. If the dataset contains highly skewed keys, a custom partitioner may be required.

A custom partitioner can distribute data more evenly based on characteristics such as:

- Key ranges
- Geographic regions
- Time periods
- Hash buckets

The goal is to ensure that no single Reducer receives a disproportionate amount of data.

Corrective Action 4: Increase or Adjust Reducer Count

The number of Reducers should be selected according to the intermediate data size and available cluster resources.

Having too few Reducers can create a bottleneck because each Reducer receives too much data.

However, simply increasing the number of Reducers does not solve data skew. If the partitioning function sends most records to one Reducer, additional Reducers may remain idle.

Therefore, **Reducer count and partitioning strategy must be considered together.**

Corrective Action 5: Use a Combiner

A Combiner can reduce the amount of intermediate data transferred from Mappers to Reducers.

For example, if the Mapper generates:

Error → 1

thousands of times, the Combiner can locally convert them into:

Error → 10,000

This reduces network traffic and the amount of work performed by Reducers.

Corrective Action 6: Handle Skewed Keys

If one key occurs much more frequently than other keys, the key can be modified by adding a random or controlled suffix.

For example:

India-1

India-2

India-3

These modified keys can be distributed across different Reducers and later combined during a final aggregation stage.

This technique is useful when a small number of keys dominate the dataset.

Corrective Action 7: Use Counters and Monitoring

Hadoop counters and monitoring tools should be used to identify:

- Number of input records
- Number of Map tasks
- Number of Reduce tasks

- Map task execution time
- Reduce task execution time
- Shuffle volume
- Records processed per Reducer

If one task takes significantly longer than the others, it indicates workload imbalance.

Corrective Architecture

The corrected workflow should be:

**Raw Data → Balanced HDFS Files → Proper Input Splits → Parallel Mappers → Combiner →
Balanced Partitioner → Reducers → HDFS Output**

This design improves both Map and Reduce parallelism.

- The repeated Hadoop job failures are most likely caused by uneven input splits, poor partitioning, or data skew. The resulting workload imbalance causes some Mapper and Reducer tasks to process significantly more data than others, while some Reducers remain idle. The problem can be corrected by reorganizing input files, selecting suitable split sizes, using data-local processing, applying a suitable partitioner, balancing Reducer count, using Combiners, and handling skewed keys. Monitoring task execution and Hadoop counters should also be used to verify that workloads are evenly distributed. Thus, proper partitioning and workload balancing are essential for reliable and efficient Hadoop MapReduce execution.

2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

ANS:

- HDFS is designed to provide reliable storage even when individual machines fail. Since Hadoop runs on commodity hardware, hardware failures are expected. HDFS therefore uses replication to maintain multiple copies of data blocks on different DataNodes.
- If data becomes unavailable whenever a single DataNode fails, there is a serious problem in the replication or NameNode architecture. A properly configured HDFS cluster should normally continue serving data when one DataNode fails, provided that another valid replica exists.

Identifying the Probable Design Error

The most likely error is that the data has an insufficient replication factor or that replicas are not correctly distributed.

For example, if the replication factor is set to **1**, each block has only one copy.

Suppose:

Block A → DataNode 1

If DataNode 1 fails, Block A becomes unavailable.

This defeats the main purpose of HDFS replication.

For important datasets, the replication factor should generally be configured appropriately, commonly up to three depending on reliability and storage requirements.

Replication Factor Problem

Consider the following configuration:

Block B → DataNode 1

Block C → DataNode 2

Block D → DataNode 3

If each block has only one copy, failure of the corresponding DataNode causes immediate unavailability.

With replication factor 3:

Block B → DataNode 1, DataNode 3, DataNode 5

the failure of DataNode 1 does not make Block B unavailable because DataNode 3 or DataNode 5 can provide it.

Therefore, the first step is to verify the replication factor of the affected files.

Incorrect Replica Placement

Even if the replication factor is three, poor replica placement can cause availability problems.

For example:

Rack 1 → DataNode 1

Rack 1 → DataNode 2

Rack 1 → DataNode 3

If all three replicas are placed in the same rack, a rack-level failure could make all copies unavailable.

HDFS should use **rack awareness** so that replicas are distributed across different racks.

A better design is:

Rack 1 → DataNode 1

Rack 2 → DataNode 2

Rack 3 → DataNode 3

This provides protection against both individual node failures and larger infrastructure failures.

NameNode High Availability Problem

Another possible design error is the absence of NameNode High Availability.

The NameNode maintains HDFS metadata, including:

- File names
- Directory structure

- Permissions
- Block locations
- Replication information

If the only NameNode fails, clients may be unable to determine where files are located even if the DataNodes themselves are healthy.

Therefore, the cluster should use:

Active NameNode + Standby NameNode

The Standby NameNode maintains synchronized metadata and can take over if the Active NameNode fails.

Automatic Failover

High availability should include automatic failover.

Coordination services such as ZooKeeper can help determine which NameNode is active and coordinate failover.

The architecture becomes:

Client → Active NameNode

and:

Standby NameNode → synchronized metadata

If the Active NameNode fails:

Failure detection → Standby promotion → Client operations continue

This reduces downtime.

DataNode Failure Detection

The NameNode continuously receives heartbeat messages from DataNodes.

If a DataNode stops sending heartbeats, it is considered unavailable.

The NameNode then:

1. Marks the DataNode as failed.
2. Stops using it for new operations.
3. Identifies blocks that have insufficient replicas.
4. Schedules re-replication.
5. Creates replacement copies on healthy DataNodes.

This automatic re-replication is important for maintaining the desired replication level.

Possible Configuration Error

Another possible cause is that administrators believe replication is configured globally when some files have lower replication levels.

For example, the cluster may have a default replication factor of three, but individual files may have been created with a replication factor of one.

Therefore, administrators should verify replication at the file and directory level.

Corrective Actions

The following actions should be implemented:

1. Verify replication factor

Check whether important files have sufficient replicas.

2. Enable rack awareness

Ensure replicas are distributed across independent racks.

3. Configure NameNode HA

Use Active and Standby NameNodes.

4. Enable automatic failover

Ensure the Standby NameNode can take over automatically.

5. Monitor DataNodes

Use heartbeat monitoring to detect node failures.

6. Enable automatic re-replication

When a DataNode fails, missing replicas should be recreated.

7. Monitor under-replicated blocks

Administrators should regularly check for blocks with fewer replicas than expected.

Example Corrected Architecture

A reliable HDFS deployment could contain:

- 2 NameNodes
- Multiple DataNodes
- 3-way replication
- Multiple racks
- ZooKeeper-based coordination
- Automatic failover
- Automatic re-replication
- Regular health monitoring
- Independent backup for disaster recovery
- Frequent data unavailability following a single DataNode failure indicates a serious fault-tolerance design problem. The most probable causes are a replication factor of one, insufficient replicas, poor replica placement, lack of rack awareness, or a single point of failure at the NameNode. The solution is to use appropriate replication, distribute replicas across racks, configure Active/Standby

NameNodes, enable automatic failover, monitor DataNode health, and automatically re-replicate under-replicated blocks. With these mechanisms, the failure of one DataNode should not result in data unavailability, and the HDFS cluster can continue operating reliably.

3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

ANS:

- Apache NiFi is widely used for data ingestion because it can collect, route, transform, and deliver data from multiple sources. ETL pipelines can then clean and transform the data before loading it into an analytics database or data lake.

- A common problem in such systems is the creation of duplicate records. Duplicate data can produce incorrect reports, inaccurate statistics, increased storage consumption, and unreliable machine learning results.
- The problem usually occurs because the ingestion pipeline does not correctly track already processed data, retries operations without idempotency, or lacks a unique-key and deduplication mechanism.

Likely Cause 1: Repeated File Ingestion

One common cause is that NiFi repeatedly reads the same file.

For example:

customers.csv

may be picked up by a NiFi flow more than once.

If the workflow simply reads the file and inserts all records into the analytics store without checking whether it was previously processed, duplicate records will be created.

Cause 2: Retry Without Idempotency

NiFi may retry a failed operation.

Suppose the data is successfully inserted into the database but the acknowledgement is lost because of a temporary network problem.

NiFi may assume that the operation failed and retry it.

The same record can then be inserted twice.

This is known as a **non-idempotent ingestion problem**.

An ingestion operation should be designed so that repeating it does not create duplicate results.

Cause 3: Lack of Unique Key

The analytics store may not have a unique constraint.

For example, suppose every customer record contains:

Customer_ID = C101

If the destination table allows multiple records with C101, the same customer can be inserted repeatedly.

A unique key or primary key should therefore be defined wherever appropriate.

Cause 4: Full Extraction Instead of Incremental Extraction

Operational databases may be extracted using a full-table query every time.

For example:

```
SELECT * FROM customers
```

If this entire table is loaded every hour, all existing records may be inserted again.

Instead, the ETL pipeline should use incremental extraction based on:

- Last modified timestamp
- Sequence number

- Change Data Capture
- Transaction log

Cause 5: Multiple NiFi Flows

Another possibility is that two NiFi flows are reading from the same source.

For example:

Flow A → Analytics Store

and:

Flow B → Analytics Store

If both flows process the same records, duplicates can occur.

Therefore, data ownership and flow design must be clearly defined.

Cause 6: Duplicate Records at Source

The source itself may contain duplicate records.

For example:

Order 1001

may appear twice in the source system.

Therefore, the ingestion process should distinguish between:

- Source duplicates
- Pipeline-generated duplicates

Corrective Action 1: Use Unique Identifiers

Each record should have a stable unique identifier.

For example:

Order_ID

or:

Transaction_ID

The destination system should enforce uniqueness.

If an incoming record already exists, it should be updated or ignored rather than inserted again.

Corrective Action 2: Implement Deduplication

NiFi can perform duplicate detection using attributes and record-level processing.

A record can be assigned a unique identifier or hash.

For example:

Hash = SHA-256(Customer_ID + Transaction_Date + Amount)

If the same hash has already been processed, the record can be routed away from the normal insertion path.

Corrective Action 3: Track File Processing

For file-based ingestion, the system should maintain metadata such as:

- File name
- File size
- File checksum
- Creation time
- Processing status
- Ingestion timestamp

A checksum is especially useful because two files may have different names but identical content.

Corrective Action 4: Use Incremental Loading

Instead of repeatedly loading the entire source table, the ETL process should extract only new or modified records.

For example:

WHERE last_updated > last_successful_run

This reduces both processing cost and duplication.

Corrective Action 5: Use Upsert Logic

The destination should support **upsert** operations where appropriate.

An upsert means:

- Insert if the record does not exist.
- Update if the record already exists.

For example:

Customer_ID = C101

If C101 already exists, the incoming record updates the existing record rather than creating a second copy.

Corrective Action 6: Maintain Checkpoints

The pipeline should maintain a checkpoint representing the last successfully processed record or file.

For database ingestion, this could be:

last_processed_timestamp

or:

last_processed_sequence_number

After successful processing, the checkpoint is updated.

If the pipeline fails and restarts, it can continue from the correct position rather than reprocessing the entire dataset.

Corrective Architecture

The corrected architecture should be:

Source → NiFi → Validation → Unique ID Generation → Duplicate Check → Transformation → Destination

A failure-management branch can handle rejected or duplicate records:

Duplicate → Duplicate/Quarantine Storage

This allows administrators to investigate duplicates without inserting them into the analytics system.

Monitoring

The pipeline should monitor:

- Records received
- Records inserted
- Records updated
- Duplicate records
- Failed records
- Processing time
- Retry count

A sudden increase in duplicate records can then be detected quickly.

- Duplicate records in an ETL and Apache NiFi pipeline can result from repeated file ingestion, retry operations, missing unique keys, full-table extraction, multiple ingestion flows, or duplicates already present at the source. The problem can be corrected through unique identifiers, destination constraints, incremental extraction, checkpoints, deduplication, file checksums, and idempotent processing. Upsert operations can also ensure that repeated records update existing data instead of creating additional copies. Therefore, a properly designed ingestion pipeline must guarantee that each logical record is processed exactly once or that repeated processing produces the same final result without duplication.

4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

ANS:

- YARN is responsible for managing computational resources in a Hadoop cluster. It allocates CPU, memory, and other resources to applications through containers. In a multi-user environment, many users may submit MapReduce, Spark, Hive, machine learning, and ETL jobs simultaneously.
- Resource starvation occurs when one or a few applications consume most of the available resources, leaving insufficient CPU or memory for other users. As a result, some jobs remain in the waiting state for a long time.
- This problem usually indicates an inappropriate scheduler configuration or absence of resource quotas and queue management.

Identifying the Scheduling Error

The most probable scheduling error is that all applications are being submitted to a single queue without appropriate resource limits.

For example:

User A → Large ML job → 70% cluster resources

User B → ETL job → Waiting

User C → Reporting job → Waiting

If the scheduler allows User A's application to consume most of the cluster, other users experience resource starvation.

Lack of Queue Isolation

A major problem is the absence of logical queues.

A better design is to divide the cluster into queues such as:

- ETL
- Reporting
- Machine Learning
- Development

Each queue receives a guaranteed capacity.

For example:

- ETL → 40%
- Reporting → 20%
- Machine Learning → 30%
- Development → 10%

These values can be adjusted according to business requirements.

Capacity Scheduler

The **Capacity Scheduler** is a suitable approach for enterprise environments where multiple groups require guaranteed resources.

Each queue receives a configured capacity.

If the ETL queue has a guaranteed 40% capacity, other workloads cannot permanently consume all of those resources.

Unused resources can be temporarily shared with other queues, improving overall utilization.

Fair Scheduler Concept

A fair scheduling approach attempts to distribute resources fairly among competing applications or users.

For example, if five users submit jobs simultaneously, resources should not be monopolized by the first user.

Fairness can be applied at:

- User level
- Queue level
- Application level

This prevents long-running jobs from blocking all other workloads.

Priority Management

Not all workloads have the same business importance.

For example:

Priority 1: Production ETL

Priority 2: Business reporting

Priority 3: Machine learning experiments

Priority 4: Development jobs

The scheduler should assign higher priority to critical production workloads.

However, priority should not completely starve lower-priority applications.

User Resource Limits

Each user should have resource limits.

For example:

- Maximum memory allocation
- Maximum CPU allocation
- Maximum number of concurrent applications
- Maximum queue capacity

Suppose User A submits ten large ML jobs. Without limits, those jobs could consume the entire ML queue.

A user-level application limit prevents this situation.

Application Limits

Applications should also have maximum resource limits.

A single application should not be allowed to consume more than a defined percentage of the cluster unless it has special authorization.

This is particularly important for large machine learning jobs.

Resource Requests

Users should specify realistic CPU and memory requirements.

If an application requests excessive resources, the scheduler may reserve large amounts of capacity even when the application does not actually need them.

Therefore, resource requests should be based on actual workload requirements.

Dynamic Resource Sharing

The scheduler should allow unused resources to be borrowed.

For example:

If the Reporting queue is using only 10% of its 20% allocation, the unused 10% can temporarily be used by ETL or ML workloads.

When reporting demand increases, the scheduler can adjust resource allocation.

This improves overall cluster utilization while maintaining fairness.

Monitoring and Autoscaling

YARN monitoring should track:

- Queue utilization
- Application waiting time

- CPU usage
- Memory usage
- Number of running applications
- Number of pending applications

If the cluster consistently reaches maximum capacity, additional worker nodes may be added.

This provides horizontal scalability.

Example Corrected Design

The improved architecture can be:

Root Queue

- Production ETL: 40%
- Reporting: 20%
- Machine Learning: 30%
- Development: 10%

Each queue has:

- Minimum capacity
- Maximum capacity
- User limits
- Application limits
- Priority settings

This creates predictable resource allocation.

Handling Starvation

Suppose an ML job requires 60 GB memory while the cluster has limited memory.

Instead of allowing the ML job to consume all resources, the scheduler limits it according to its queue capacity.

At the same time, ETL and reporting workloads receive their guaranteed allocations.

This allows all categories of jobs to make progress.

- YARN resource starvation is usually caused by poor scheduling configuration, a single shared queue, lack of user limits, excessive application resource requests, or inappropriate priority settings. The problem can be corrected by implementing separate queues, guaranteed capacity, fair resource sharing, priority management, user and application limits, and dynamic resource allocation. The Capacity Scheduler is particularly suitable for enterprise environments requiring predictable resource guarantees. Continuous monitoring and cluster scaling should also be used when demand exceeds available capacity. Thus, proper YARN resource management ensures that multiple users

can run workloads simultaneously without allowing one user or application to monopolize the cluster.

- 5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.**

ANS:

- Distributed storage systems use replication and backup mechanisms to protect data against hardware failure, software errors, accidental deletion, and disaster scenarios. However, replication and backup are not identical concepts. Replication maintains multiple copies of data, while backup preserves recoverable historical versions.
- If a recovery operation restores outdated data, the backup architecture is likely using stale backups, improperly synchronized replicas, or an incorrect recovery point.
- The main design problem is that the system has not correctly maintained the relationship between the latest production data and the backup copy used during recovery.

Difference Between Replication and Backup:

Replication provides multiple current copies of data.

For example:

Primary → Replica 1 → Replica 2

If the primary copy is updated, replication mechanisms attempt to propagate the change.

Backup, on the other hand, creates recoverable copies at specific points in time.

For example:

- Monday backup
- Tuesday backup
- Wednesday backup
- Thursday backup

If the system fails on Friday but only the Monday backup is available, recovery will restore Monday's version.

Therefore, replication alone is not sufficient for complete backup protection.

Probable Design Flaw 1: Stale Backup

The most likely problem is that the backup schedule is too infrequent.

For example:

Production data updated daily

but:

Backup performed once per week

If the system fails before the next backup, the recovery process may restore the previous week's data.

This creates a large Recovery Point Objective (RPO).

Recovery Point Objective

RPO defines how much recent data the organization can afford to lose.

For example:

If:

RPO = 1 hour

the organization should be able to recover data to approximately one hour before failure.

If backups occur only once every 24 hours, the system cannot satisfy a one-hour RPO.

Therefore, the backup frequency must match business requirements.

Probable Design Flaw 2: Treating Replicas as Backups

Another common mistake is assuming that HDFS replicas are equivalent to independent backups.

Suppose a file has three replicas:

DataNode 1

DataNode 2

DataNode 3

If an application accidentally deletes or corrupts the file, the change may affect the replicated copies as well.

Replication protects mainly against infrastructure failures such as DataNode failure. It does not necessarily protect against:

- Accidental deletion
- Logical corruption

- Malicious modification
- Application errors

Therefore, an independent backup system is necessary.

Probable Design Flaw 3: No Versioning

If backups overwrite the previous backup instead of maintaining versions, recovery may not be able to return to an earlier correct state.

For example:

Backup Monday → overwritten

Backup Tuesday → overwritten

Backup Wednesday → current backup

If corruption occurs on Thursday, the system may not have a clean earlier version.

Versioned backups allow recovery to multiple historical points.

Probable Design Flaw 4: Backup Synchronization Failure

Another possibility is that the backup system is not synchronized with the production environment.

Suppose:

Production database → HDFS

but the backup process executes before the latest ingestion job finishes.

The backup may contain only part of the latest dataset.

Therefore, backup operations should be coordinated with data ingestion and processing jobs.

Corrective Action 1: Define RPO and RTO

The organization should define:

RPO: Maximum acceptable data loss.

RTO: Maximum acceptable recovery time.

For example:

RPO = 1 hour

RTO = 2 hours

The backup architecture should then be designed to meet these requirements.

Corrective Action 2: Use Incremental Backups

Full backups every time can be expensive.

Instead:

Periodic Full Backup + Frequent Incremental Backups

can be used.

For example:

- Sunday → Full backup
- Monday–Saturday → Incremental backups

This reduces storage consumption and backup time.

Corrective Action 3: Maintain Multiple Backup Versions

Backups should be versioned.

For example:

- Daily recovery points
- Weekly recovery points
- Monthly archival points

This allows administrators to select the correct recovery version.

Corrective Action 4: Store Backups Independently

Backups should not be stored only on the same physical infrastructure as the production system.

A stronger architecture is:

Primary Hadoop Cluster

↓

Backup System

↓

Secondary Cluster / Independent Storage / Off-site Storage

This protects against large-scale infrastructure failure.

Corrective Action 5: Verify Backup Integrity

A backup should not be considered reliable merely because the backup job completed successfully.

The system should periodically perform recovery tests.

For example:

1. Select a backup.
2. Restore it into a test environment.
3. Verify file integrity.
4. Compare expected record counts.
5. Validate important datasets.
6. Record recovery time.

This ensures that the backup is actually usable.

Corrective Action 6: Use Snapshots and Versioning

Snapshots can provide point-in-time recovery.

If data is accidentally modified or deleted, the organization can recover the previous version.

Snapshots should be combined with independent backups because snapshots alone may not protect against every disaster scenario.

Corrected Backup Architecture

A reliable design is:

Production HDFS

- Replicated DataNodes
- Periodic Snapshots
- Independent Backup Storage
- Versioned Full + Incremental Backups
- Regular Recovery Testing

This provides multiple layers of protection.

Example

Suppose a company's production data is updated continuously.

A suitable strategy could be:

- HDFS replication for DataNode failure
- Hourly incremental backup for recent changes
- Daily full backup
- Weekly long-term backup
- Independent disaster-recovery copy
- Periodic restore testing

If corruption occurs at 3:00 PM, the organization can restore the most recent valid backup instead of reverting to an outdated weekly copy.

- Outdated data after recovery usually indicates stale backups, excessive backup intervals, lack of versioning, failure to synchronize backups with production data, or the incorrect assumption that replication itself provides complete backup protection. The solution is to define appropriate RPO and RTO values, perform regular incremental and full backups, maintain multiple versions, store backups independently, use snapshots where appropriate, and regularly test restoration. Replication should be used for high availability and hardware failure protection, while independent versioned

backups should be used for disaster recovery and protection against logical data loss. This layered approach ensures that recovery restores the most recent valid version of the data rather than an outdated copy.